

Figure 5.12 Updating or modifying the document is a three-step process.

If you think we can make the same updates by calling the `GET`, `UPDATE`, and `POST` methods on the document individually, you are absolutely right. In reality, this is what Elasticsearch does. As you can imagine, these are three different calls to the server, each resulting in a round trip to and from the client to the server. Elasticsearch avoids this round trip by cleverly executing the set of operations on the same shard, thus saving network traffic between client and server. By using the `_update` API, Elasticsearch avoids network calls, bandwidth, and coding errors.

5.4.2 The _update API

When we plan to update a document, we usually focus on one or more of the following scenarios:

- Adding more fields to an existing document
- Modifying existing field values
- Replacing the whole document

All of these operations are performed using the `_update` API. The format is simple:

```
POST <index_name>/_update/<id>
```

When managing our resources with the `_update` API, we use the `POST` method based on RESTful API conventions. The document ID is provided in the URL along with the index name to form the full URL.

> **NOTE** The previous update URL uses the `_update` endpoint, but we can also use `POST <index_name>/_doc/<id>/_update`. Because the types were deprecated and are removed from version 8, avoid using the `_doc` endpoint to update the document.

Now that we know the basics of the `_update` API, let's update our `movie` document (*The Godfather*) with a couple of additional attributes: `actors` and `director`.

ADDING NEW FIELDS

To amend a document with new fields, we pass a request body consisting of the new fields to the `_update` API. The new fields are wrapped in a `doc` object as the API expects. The code in the following listing amends the `movie` document with two additional fields. Noted that we are implicitly assuming that `dynamic=true`.